

LikeALocal — Flutter Implementation Plan

AI AGENT IMPLEMENTATION GUIDE — Read every line before writing any code.

App Name	LikeALocal
Framework	Flutter (Dart) — Android only
State Management	Provider — REQUIRED by course, do not use Riverpod/Bloc
Backend	Firebase: Auth + Firestore + Storage + FCM (Cloud Messaging)
AI Feature	Claude API (claude-sonnet-4-20250514) — personalized chatbot
Maps	google_maps_flutter + geolocator
Deadline	May 18, 2026 — code zip + screen recording + in-class demo
Simulator	Android Emulator (Pixel 6, API 34) — no physical phone for now

RULES FOR THE AI AGENT — DO NOT VIOLATE

1. Use Provider for ALL state management. Never use setState except inside widgets with no shared state.
2. Every Firebase call must be inside a try/catch. Show a Snackbar with a readable error message on failure.
3. Do NOT add any feature, package, or screen not listed in this plan.
4. All user-generated text fields must have input validation before submitting.
5. Comments and reviews must support edit and delete by their author only.
6. Super User status is earned by karma — NEVER hardcoded.
7. The AI chatbot uses the Claude API, not any local model.
8. Privacy: check chatEnabled flag before allowing DMs to a user.
9. Match the color palette exactly: primary=#E8580A, dark=#1A1A2E, amber=#D4820A.
10. Do not invent Firestore collection names. Use exactly the names defined in the Data Models section.

■ SECTION 0: ONE-TIME SETUP (do this before any code)

TASK 0.1: Install Flutter on Ubuntu

- `sudo apt update && sudo apt install -y curl git unzip xz-utils zip libglu1-mesa openjdk-17-jdk`
- `git clone https://github.com/flutter/flutter.git -b stable ~/flutter`
- `echo 'export PATH="$PATH:$HOME/flutter/bin"' >> ~/.bashrc && source ~/.bashrc`

TASK 0.2: Install Android Studio + Emulator

- Download Android Studio from developer.android.com/studio and install it
- Open SDK Manager → install: Android SDK 34, Android SDK Command-line Tools, Android Emulator
- Open Device Manager → Create Device → Pixel 6 → API 34 → x86_64 → Finish
- Start the emulator once to confirm it boots
- Run: `flutter doctor --android-licenses` (accept all)
- Run: `flutter doctor` (must show all green before continuing)

TASK 0.3: Create Flutter Project

- `flutter create like_a_local --org com.likeALocal`
- `cd like_a_local`
- `flutter devices` (emulator must appear in list)
- `flutter run` (default counter app must launch on emulator — confirms setup works)

TASK 0.4: Set Up Git

- Inside `like_a_local/`: `git init && git add . && git commit -m "init"`
- Create repo on GitHub called `like_a_local`
- `git remote add origin https://github.com/YOUR_USERNAME/like_a_local.git`
- `git branch -M main && git push -u origin main`
- Commit rule: one commit after each completed task. Message = "feat: [task name]"

TASK 0.5: Create Firebase Project

- Go to console.firebase.google.com → Add project → name: `LikeALocal`
- Enable Authentication → Sign-in methods: Email/Password + Google
- Enable Firestore Database → Start in test mode
- Enable Storage → Start in test mode
- Enable Cloud Messaging (FCM) — no extra config needed yet
- Install FlutterFire CLI: `dart pub global activate flutterfire_cli`
- Inside project folder run: `flutterfire configure`
- This generates `lib/firebase_options.dart` — do not edit this file

TASK 0.6: Add All Packages to pubspec.yaml

- Add under dependencies: (copy exactly)

```
firebase_core: ^3.0.0
firebase_auth: ^5.0.0
cloud_firestore: ^5.0.0
firebase_storage: ^12.0.0
firebase_messaging: ^15.0.0
google_sign_in: ^6.0.0
provider: ^6.0.0
google_maps_flutter: ^2.5.0
geolocator: ^12.0.0
```

```
geocoding: ^3.0.0
image_picker: ^1.0.0
http: ^1.0.0
shared_preferences: ^2.0.0
connectivity_plus: ^6.0.0
shimmer: ^3.0.0
timeago: ^3.0.0
cached_network_image: ^3.0.0
go_router: ^14.0.0
flutter_local_notifications: ^17.0.0
```

TASK 0.7: Create Folder Structure

→ Create these folders inside lib/ (create a placeholder .dart file in each so git tracks them):

```
lib/
■■■ main.dart
■■■ firebase_options.dart ← auto-generated, do not touch
■■■ core/theme/ ← app_theme.dart, app_colors.dart
■■■ core/constants/ ← app_routes.dart
■■■ core/utils/ ← validators.dart, error_handler.dart
■■■ data/models/ ← user_model.dart, post_model.dart, comment_model.dart, message_model.dart
■■■ data/repositories/ ← auth_repo.dart, posts_repo.dart, chat_repo.dart, user_repo.dart
■■■ data/services/ ← ai_service.dart, location_service.dart, notification_service.dart
■■■ features/auth/ ← login_screen.dart, register_screen.dart
■■■ features/feed/ ← posts_screen.dart, post_detail_screen.dart
■■■ features/create/ ← create_post_screen.dart
■■■ features/map/ ← map_screen.dart
■■■ features/chat/ ← chat_screen.dart, conversation_screen.dart
■■■ features/search/ ← search_screen.dart
■■■ features/profile/ ← profile_screen.dart
■■■ features/notifications/ ← notifications_screen.dart
■■■ features/saved/ ← saved_posts_screen.dart
■■■ features/settings/ ← settings_screen.dart
■■■ shared/widgets/ ← bottom_nav.dart, top_bar.dart, post_card.dart, super_user_badge.dart,
sidebar_menu.dart
■■■ shared/providers/ ← auth_provider.dart, posts_provider.dart, chat_provider.dart, user_provider.dart
```

■ SECTION 1: DATA MODELS (implement these before any screen)

NOTE: These are the exact Firestore collection names and field names. Do not rename them.

Firestore Collection: users/{uid}

Field	Type	Notes
uid	String	Firebase Auth UID
username	String	Display name
avatarUrl	String	Firebase Storage download URL
bio	String	Profile description
location	String	e.g. Cairo, Egypt
karma	int	Starts at 0. +10 post created, +2 upvote received, +1 comment posted, +3 DM reply
isSuperUser	bool	Set to true when karma >= 1000. Affects feed ordering.
contributionScore	double	0.0–100.0 shown as progress bar. $\text{contributionScore} = (\text{karma}/1000) * 100$, capped at 100
chatEnabled	bool	true = anyone can DM this user. false = DMs blocked.
chatSchedule	Map?	Optional. Keys: daysOfWeek (List<int>), startHour (int), endHour (int)
preferences	Map	Keys: budget (String), atmosphere (String), favCategories (List<String>)
joinedAt	Timestamp	Set on registration
fcmToken	String	Updated on each app launch for push notifications

Firestore Collection: posts/{postId}

Field	Type	Notes
postId	String	Firestore auto-ID
userId	String	Author UID
username	String	Denormalized from user doc
userAvatarUrl	String	Denormalized from user doc
isSuperUser	bool	Denormalized from user doc at time of post creation
title	String	Post title
description	String	Main body text
localTips	String	Optional insider tips
recommendedDishes	List<String>	Optional, for food category
imageUrl	String	Firebase Storage download URL
location	String	Human-readable address
lat	double	Latitude
lng	double	Longitude
category	String	One of: Restaurant, Bar, Café, Park, Viewpoint, Shop
upvotes	int	Starts at 0
downvotes	int	Starts at 0
commentCount	int	Cached. Increment/decrement when comments added/deleted
createdAt	Timestamp	

Firestore Collection: posts/{postId}/comments/{commentId}

Field	Type	Notes
-------	------	-------

commentId	String	Firestore auto-ID
postId	String	Parent post ID
userId	String	Author UID
username	String	Denormalized
userAvatarUrl	String	Denormalized
isSuperUser	bool	Denormalized
content	String	Comment text
upvotes	int	
parentId	String?	null = top-level comment. set to another commentId for a reply
createdAt	Timestamp	
editedAt	Timestamp?	Set when user edits. null if never edited

Firestore Collection: chats/{chatId}

Field	Type	Notes
chatId	String	Auto-ID or "ai_{userId}" for AI chat
participants	List<String>	List of user UIDs in the chat
isGroup	bool	
groupName	String?	Only for group chats
lastMessage	String	Cached last message text
lastMessageAt	Timestamp	
unreadCount	Map<String,int>	Key = userId, value = unread count

Firestore Collection: chats/{chatId}/messages/{msgId}

Field	Type	Notes
msgId	String	Auto-ID
senderId	String	UID or "ai" for AI messages
text	String	Message content
type	String	One of: text, image, ai_response
createdAt	Timestamp	

Firestore Collection: notifications/{notifId}

Field	Type	Notes
userId	String	Who receives this notification
type	String	One of: upvote, comment, new_dm, nearby_gem, ai_recommendation
title	String	
body	String	
postId	String?	Related post if applicable
read	bool	false on creation
createdAt	Timestamp	

■ SECTION 2: THEME & COLORS (implement in core/theme/ before any screen)

TASK 2.1: Create lib/core/theme/app_colors.dart

→ Define these exact constants — they come from the Figma design:

```
import 'package:flutter/material.dart';  
const kOrange = Color(0xFFE8580A); // primary buttons, active nav, badges  
const kDark = Color(0xFF1A1A2E); // app bar, headers, primary text  
const kAmber = Color(0xFFD4820A); // Super User pins, Super User card ring  
const kBackground = Color(0xFFFFFFFF);  
const kMuted = Color(0xFFE0E0E0); // input fill, chip backgrounds  
const kMutedFg = Color(0xFF717171); // placeholder text, secondary labels  
const kDestructive = Color(0xFFD4183D); // delete buttons, error states  
const kSuperUserBg = Color(0xFFFFF8E6); // amber-tinted card background
```

TASK 2.2: Create lib/core/theme/app_theme.dart

→ ThemeData with: primaryColor=kOrange, scaffoldBackgroundColor=kBackground

→ AppBarTheme: backgroundColor=kDark, foregroundColor=Colors.white

→ ElevatedButtonTheme: backgroundColor=kOrange, foregroundColor=Colors.white, borderRadius=8

→ InputDecorationTheme: filled=true, fillColor=kMuted, border=OutlineInputBorder(borderRadius=8, borderSide=None)

→ BottomNavigationBarTheme: selectedItemColor=kOrange, unselectedItemColor=kMutedFg

→ TextTheme: use system font (no custom fonts needed)

■ SECTION 3: IMPLEMENTATION PHASES (do in order, do not skip ahead)

PHASE 1: AUTH & NAVIGATION

Days 1–2

TASK 1.1: lib/main.dart — app entry point

- Call `WidgetsFlutterBinding.ensureInitialized()`
- Call `await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)`
- Wrap app in `MultiProvider` with: `AuthProvider`, `PostsProvider`, `ChatProvider`, `UserProvider`
- Use `GoRouter` for routing. Root route `"/` checks auth state: if logged in → `"/feed"`, else → `"/login"`
- Apply `AppTheme` defined in Section 2

TASK 1.2: lib/shared/providers/auth_provider.dart

- Extend `ChangeNotifier`
- Listen to `FirebaseAuth.instance.authStateChanges()` stream
- Expose: `User? currentUser`, `bool isLoading`, `String? errorMessage`
- Methods: `signIn(email, password)`, `register(email, password, username)`, `signInWithGoogle()`, `signOut()`, `resetPassword(email)`
- On register: also create `users/{uid}` document in Firestore with all fields from Section 1
- On any error: set `errorMessage`, call `notifyListeners()`

TASK 1.3: lib/core/utils/validators.dart

- `String? validateEmail(String? val)` — must contain `@` and `.`. Return error string or null
- `String? validatePassword(String? val)` — min 6 chars
- `String? validateUsername(String? val)` — min 3 chars, no spaces
- `String? validateNotEmpty(String? val, String fieldName)` — generic non-empty check

TASK 1.4: lib/features/auth/login_screen.dart

- Background: gradient from `Color(0xFFFFF7F2)` to `Color(0xFFFFCE8E0)`
- Center: `LikeALocal` logo (stack icon in orange gradient box) + app name + tagline "Discover hidden gems in your city"
- White card with: tab toggle (Log In / Sign Up), form fields, submit button, divider, Google button, forgot password link
- Log In tab: Email field + Password field + "Log In" button
- Sign Up tab: Email + Password + Username fields + "Create Account" button
- Both tabs: "Continue with Google" button below divider
- On submit: call `AuthProvider` method. Show `CircularProgressIndicator` while loading.
- On error: show `SnackBar` with `errorMessage` from provider
- On success: `GoRouter` redirects automatically via auth state change

TASK 1.5: lib/shared/widgets/bottom_nav.dart

- `BottomNavigationBar` with 5 items: Posts (home icon), Map (map icon), Create (add_circle icon, larger), Chat (chat_bubble icon), Search (search icon)
- Create button is center, styled with `kOrange` fill to stand out
- Selected color = `kOrange`, unselected = `kMutedFg`
- Navigation: Posts→`"/feed"`, Map→`"/map"`, Create→`"/create"`, Chat→`"/chat"`, Search→`"/search"`

TASK 1.6: lib/shared/widgets/top_bar.dart

- `AppBar`: left = hamburger menu icon (opens Drawer/sidebar), center = `LikeALocal` logo + text, right = notification bell with badge + user avatar
- Notification badge: red circle with count on bell icon. Pull unread count from `NotificationsProvider`.

- Avatar tapped → navigate to "/profile"
- Background = kDark, foreground = white

TASK 1.7: lib/shared/widgets/sidebar_menu.dart (Drawer)

- Slide-in Drawer triggered by hamburger icon in TopBar
- Header: user avatar (circle) + username + karma score (e.g. "■ 1,842 karma")
- Below header: progress bar "Path to Super User" — LinearProgressIndicator value=contributionScore/100, color=kAmber
- Label: "X% — Y more to unlock Super User status" where Y = 1000 - karma
- Menu items (icon + label): Profile, Saved Posts, Notifications, Settings
- Each item tapped → navigate to its route and close drawer
- Pull user data from UserProvider (current user doc)

TASK 1.8: lib/shared/widgets/super_user_badge.dart

- Small widget: amber icon (star or crown) + text "Super User" in amber color
- Used inside PostCard and comment widgets
- Only render if isSuperUser == true

NOTE: Phase 1 done when: user can register, log in (email or Google), see the feed shell with BottomNav and TopBar, and log out via Settings. Commit: "feat: phase 1 auth and navigation"

PHASE 2: FEED, POST DETAIL, CREATE POST & MAP

Days 3–6

TASK 2.1: lib/data/repositories/posts_repo.dart

- Stream> getFeedPosts() — Firestore query: collection("posts"), orderBy("isSuperUser", descending:true), orderBy("createdAt", descending:true)
- Future createPost(PostModel post) — add to Firestore, upload image to Storage first
- Future upvotePost(postId, userId) — increment upvotes, also +2 karma to post author
- Future downvotePost(postId, userId) — increment downvotes
- Future savePost(userId, postId) — write to users/{userId}/savedPosts/{postId}
- Future unsavePost(userId, postId) — delete from savedPosts
- Future isPostSaved(userId, postId) — check if savedPosts doc exists

TASK 2.2: lib/shared/providers/posts_provider.dart

- Extend ChangeNotifier
- List feedPosts — updated via stream subscription
- bool isLoading
- Call postsRepo.getFeedPosts() stream in init, update feedPosts on each emission
- Methods: createPost, upvotePost, downvotePost, savePost, unsavePost

TASK 2.3: lib/shared/widgets/post_card.dart

- Card widget used in feed and search results
- If post.isSuperUser == true: wrap card in amber border (Border.all(color:kAmber, width:1.5))
- Layout top-to-bottom:
 - Row: user avatar (CircleAvatar) + username + SuperUserBadge (if super user) + location + timeAgo
 - Title text (bold)
 - Description text (2 lines max, overflow ellipsis)
 - Post image (CachedNetworkImage, full width, height 200)
 - Action row: upvote button (+score) + downvote button | comment count icon | bookmark icon | share icon
- Upvote/downvote: toggle state locally, call provider method. Orange = active.

→ Bookmark: toggle, call provider savePost/unsavePost

→ Tap anywhere on card → navigate to "/post/{postId}"

TASK 2.4: lib/features/feed/posts_screen.dart

→ StreamBuilder or Consumer to get feedPosts list

→ Show shimmer loading placeholders while isLoading == true (use shimmer package)

→ ListView.builder rendering PostCard for each post

→ Pull-to-refresh: RefreshIndicator wrapping the list

→ Empty state: centered icon + text "No posts yet. Be the first to share a hidden gem!"

TASK 2.5: lib/data/repositories/posts_repo.dart — comments section

→ Stream> getComments(postId) — query subcollection, orderBy createdAt asc

→ Future addComment(CommentModel) — also increment posts/{postId}.commentCount by 1, +1 karma to commenter

→ Future editComment(postId, commentId, newContent) — update content + set editedAt

→ Future deleteComment(postId, commentId) — delete doc, decrement commentCount by 1

TASK 2.6: lib/features/feed/post_detail_screen.dart

→ Hero animation on the post image (tag = postId)

→ Show: full image, title, description, location, category chip, localTips, recommendedDishes list

→ Comments section below: StreamBuilder on getComments(postId)

→ CommentItem widget per comment:

→ → Avatar + username + SuperUserBadge + timeAgo

→ → Content text

→ → If editedAt != null: show "(edited)" label

→ → If comment.userId == currentUser.uid: show 3-dot menu with Edit and Delete options

→ → Edit: show TextField pre-filled with content. Save/Cancel buttons. Call editComment.

→ → Delete: show AlertDialog "Delete comment?" with Confirm/Cancel. Call deleteComment.

→ Replies: indent reply comments under parent (show if parentId != null)

→ Add comment bar pinned to bottom: TextField + send icon button. Calls addComment.

→ "Message owner" button: if post.userId != currentUser.uid AND post owner has chatEnabled==true → navigate to "/conversation/{chatId}"

TASK 2.7: lib/features/create/create_post_screen.dart

→ AppBar with "Cancel" (left) and "Post" button (right, kOrange)

→ Image picker area: dashed border box with image icon + "Add a photo". Tap → ImagePicker().pickImage(). Show selected image preview.

→ Caption: multiline TextField, hint "Share what makes this place special..."

→ Location: TextField with pin icon, hint "Where is this hidden gem?"

→ Type of Hidden Gem: GridView 2-column with buttons: Restaurant, Bar, Café, Park, Viewpoint, Shop. Selected = kOrange background.

→ Local Tips: optional TextField

→ Recommended Dishes: optional TextField (comma-separated, split into list)

→ On Post tapped: validate all required fields. Upload image to Storage at posts/{postId}/image.jpg. Write PostModel to Firestore. Add +10 karma to current user. Navigate back.

→ Error handling: show SnackBar if image upload fails or Firestore write fails

TASK 2.8: lib/features/map/map_screen.dart

→ Full-screen GoogleMap widget

→ On init: get current GPS position via geolocator, center map there

→ Load all posts from PostsProvider that have lat/lng set

→ Add custom marker per post:

→ → isSuperUser == true: amber star marker (BitmapDescriptor custom icon)

→ → isSuperUser == false: orange pin marker

→ Top overlay: search TextField + navigation button (centers map on user location)

→ Below search: horizontal filter chips: All, Food, Cafes, Parks, Art, Shopping. Filtering changes which markers are shown.

→ Tap marker → show bottom sheet (DraggableScrollableSheet) with: post image thumbnail + title + location + "View Post" button + "Pin for Later" button

→ "Pin for Later" calls savePost. Show SnackBar "Pinned!"

→ Legend at bottom-left: orange dot "Regular Post", amber star "Super User Post"

NOTE: Phase 2 done when: feed shows live posts from Firestore, post detail with edit/delete comments works, create post uploads to Firebase, map shows markers. Commit: "feat: phase 2 feed map create"

TASK 3.1: lib/data/repositories/chat_repo.dart

- Stream> getUserChats(userId) — query chats where participants array-contains userId, orderBy lastMessageAt desc
- Stream> getMessages(chatId) — subcollection messages, orderBy createdAt asc
- Future getOrCreateChat(currentUserId, otherUserId) — check if chat with these 2 participants exists, create if not, return chatId
- Future sendMessage(chatId, MessageModel) — add to messages subcollection, update chat.lastMessage and lastMessageAt
- Future markRead(chatId, userId) — set unreadCount.{userId} = 0

TASK 3.2: lib/features/chat/chat_screen.dart (Chat List)

- Header: "Messages" title + hamburger icon (opens FriendsSidebar)
- Search bar: TextField "Search conversations..."
- AI Discovery Assistant row always first: gradient purple/pink avatar with sparkle icon + name + last message + time
- Then DM conversations: avatar + online status dot + name + last message + unread count badge + time
- Then group chats: stacked avatars or group icon + name + last message + unread badge
- Tap any row → navigate to "/conversation/{chatId}"
- Pull data from ChatProvider (stream of user chats)

TASK 3.3: lib/features/chat/conversation_screen.dart

- AppBar: other user avatar + name + online status OR "AI Discovery Assistant" with sparkle icon
- Chat bubble list: StreamBuilder on getMessages(chatId), ListView.builder reversed=true
- → My messages: right-aligned, kOrange background, white text
- → Other messages: left-aligned, kMuted background, dark text
- → AI messages: left-aligned, purple/gradient background, show sparkle icon
- → Show timestamp below each bubble
- Input bar pinned to bottom: TextField + send icon button
- If this is the AI chat (chatId starts with "ai_"): send message to Claude API via AIService, add AI response as new message
- If this is a DM: check that the other user has chatEnabled==true. If false, show banner "This user has disabled messages".
- On send: call chatRepo.sendMessage(), update lastMessage in chat doc, add karma +3 to current user

TASK 3.4: lib/data/services/ai_service.dart

- Method: Future getAIResponse(List history, UserModel currentUser)
- Build system prompt:

```
You are the AI Discovery Assistant for LikeALocal, a local travel app.
The user's preferences: budget=${user.preferences['budget']},
atmosphere=${user.preferences['atmosphere']},
favorite categories=${user.preferences['favCategories']}.
Help them discover hidden gems, restaurants, and local experiences.
Be friendly, concise, and always suggest specific types of places.
```

TASK 3.4 cont.: ai_service.dart continued

- Build messages array from last 10 messages in history
- POST to https://api.anthropic.com/v1/messages with headers: x-api-key, anthropic-version: 2023-06-01, content-type: application/json
- Body: { model: "claude-sonnet-4-20250514", max_tokens: 500, system: systemPrompt, messages: [...] }
- Parse response: data["content"][0]["text"]
- Wrap in try/catch. On error return "Sorry, I'm having trouble connecting. Please try again."

→ IMPORTANT: Store the API key in a config file not committed to git. Add to .gitignore.

TASK 3.5: lib/features/chat/friends_sidebar.dart (side drawer on chat screen)

- Slide-in panel from left (use Drawer or custom overlay)
- Header: "Friends" + X close button
- Search bar: "Search friends..."
- Section "X Online": list users with green dot
- Section offline users: grey dot
- Each row: avatar + username + online/offline label
- Bottom: "Add Friend" button (tap → show dialog with username input field)
- Pull friends list from UserProvider

TASK 3.6: lib/features/settings/settings_screen.dart — Chat Privacy section

- Section "ACCOUNT": Edit Profile (navigate to profile edit), Privacy & Security
- Section "NOTIFICATIONS": Push Notifications toggle, Nearby Gems toggle, Comments & Replies toggle
- → All toggles backed by shared_preferences AND Firestore users/{uid} fields
- Section "PREFERENCES": Language (display only "English" for now), Location Services
- Section "SUPPORT": Help Center, About
- Chat Privacy (inside Privacy & Security): SwitchListTile "Allow Messages" — reads/writes chatEnabled in Firestore
- Footer: "LikeALocal v1.0.0"

NOTE: Phase 3 done when: chat list shows conversations, DMs work in real time, AI assistant responds with Claude API, friends sidebar opens. Commit: "feat: phase 3 chat and AI"

PHASE 4: SUPER USER, SEARCH, PROFILE & SAVED

Days 10–12

TASK 4.1: Karma & Super User logic (in user_repo.dart)

- Future addKarma(userId, int amount) — increment users/{userId}.karma by amount
- After each karma update: recalculate contributionScore = min((karma/1000)*100, 100)
- If new karma >= 1000 AND isSuperUser == false: set isSuperUser = true
- Call addKarma in: createPost (+10), upvote received (+2), addComment (+1), sendMessage DM reply (+3)
- Super User feed ordering is automatic — isSuperUser field on post is denormalized and Firestore orders by it

TASK 4.2: lib/features/search/search_screen.dart

- Sticky search bar at top: TextField "Search hidden gems..."
- Category filter chips row: All , Food , Cafes , Culture , Nature . Selected chip = kOrange.
- When search bar is EMPTY show:
 - → Section "Trending Searches": 6 tag chips (hardcoded): Hidden rooftop bars, Local bakeries, Secret gardens, Vintage bookstores, Street art spots, Jazz clubs
 - → Section "Recent": list of last 5 searches (stored in shared_preferences)
 - → Section "Recommended for You": top 3 posts filtered by user.preferences.favCategories
- When search bar has text: filter PostsProvider.feedPosts by title/description/location containing query AND selected category
- Results: ListView of PostCard widgets
- On search submitted: save query to recent searches in shared_preferences
- Empty results state: "No hidden gems found. Try a different search."

TASK 4.3: lib/features/profile/profile_screen.dart

- AppBar: "Profile" title + settings gear icon (→ settings screen)
- Profile card: CircleAvatar (large) + username + karma badge " X karma" + location + join date

→ Bio text below

→ Stats row: Posts count | Followers count | Following count (tap each → future feature, show toast for now)

→ Path to Super User card: LinearProgressIndicator (value=contributionScore/100, color=kAmber) + percentage text + motivational label

→ TabBar: "My Posts" | "Saved"

→ → My Posts tab: GridView of user posts (image thumbnail). Tap → post detail.

→ → Saved tab: same GridView but from savedPosts subcollection. Freemium gate: if savedPosts.length > 5 AND user is not premium, show lock overlay on 6th item with "Upgrade to Premium" text.

→ Pull data from UserProvider.currentUser and PostsProvider

TASK 4.4: lib/features/saved/saved_posts_screen.dart

→ Shows posts saved by current user (from users/{uid}/savedPosts)

→ GridView with PostCard thumbnails

→ Freemium gate: if count > 5, show AlertDialog "You have reached your free limit of 5 pins. Upgrade to Premium for unlimited saves." with Upgrade and Cancel buttons

→ Upgrade button: show SnackBar "Premium coming soon!" (placeholder)

→ Long press on saved post: show "Remove from saved" option

NOTE: Phase 4 done when: karma increments on actions, Super User badge appears on high-karma users, search works with categories and recommendations, profile shows progress bar, saved posts has freemium gate. Commit: "feat: phase 4 super user search profile"

TASK 5.1: lib/data/services/notification_service.dart

- On app start: request FCM permission (FirebaseMessaging.instance.requestPermission)
- Get FCM token and save to users/{uid}.fcmToken in Firestore
- FirebaseMessaging.onMessage.listen — handle foreground notifications: show flutter_local_notifications banner
- FirebaseMessaging.onBackgroundMessage — static handler for background/killed state
- FirebaseMessaging.onMessageOpenedApp — navigate to correct screen when tapped
- Notification types to handle:
 - → upvote: "Someone upvoted your post [title]" → navigate to post detail
 - → comment: "X commented on your post" → navigate to post detail
 - → new_dm: "New message from X" → navigate to conversation
 - → nearby_gem: "Hidden gem nearby: [title]" → navigate to map
- Write notification doc to Firestore notifications/ collection on each triggered event

TASK 5.2: lib/features/notifications/notifications_screen.dart

- AppBar: "Notifications" + "Mark all as read" button
- StreamBuilder on notifications/ where userId == currentUser.uid, orderBy createdAt desc
- Each notification row: icon (based on type) + title text + body text + timeAgo + blue dot if unread
- Tap row: mark as read (set read=true in Firestore), navigate to relevant screen
- Empty state: "No notifications yet"

TASK 5.3: Geofencing / Nearby Gems (OUT-OF-SCOPE FEATURE #1)

- Use geolocator package: Geolocator.getPositionStream() for continuous location updates
- On each location update: query Firestore posts where distance from current position < 800 meters
- → Calculate distance: use Geolocator.distanceBetween(lat1,lng1,lat2,lng2)
- → If a nearby post is found AND user has not seen it (check shared_preferences "seen_posts" list):
 - → Create notification doc in Firestore (type: nearby_gem)
 - → Show flutter_local_notifications local notification
 - → Add postId to "seen_posts" list so it does not re-trigger
- Only run when settings Nearby Gems toggle is ON

TASK 5.4: Offline Support (OUT-OF-SCOPE FEATURE #2)

- Enable Firestore offline persistence: FirebaseFirestore.instance.settings = const Settings(persistenceEnabled: true, cacheSizeBytes: Settings.CACHE_SIZE_UNLIMITED)
- Add this BEFORE any Firestore reads, in main.dart right after Firebase.initializeApp()
- Use connectivity_plus to detect network status:
 - → Create ConnectivityProvider (ChangeNotifier) that listens to Connectivity().onConnectivityChanged
 - → If no connectivity: show persistent orange banner at top "You are offline. Showing cached content."
 - → Banner disappears when connection restored
- Cache user preferences to shared_preferences so they load instantly on app open
- Cache recent searches in shared_preferences (already done in Search screen)

TASK 5.5: Error Handling (apply everywhere)

- lib/core/utils/error_handler.dart: static method showError(BuildContext ctx, String msg) → ScaffoldMessenger.of(ctx).showSnackBar(SnackBar(content: Text(msg), backgroundColor:kDestructive))
- Wrap EVERY Firebase call (read, write, auth) in try/catch

- FirebaseAuthException: map error codes to readable messages (wrong-password → "Incorrect password", user-not-found → "No account with this email", etc.)
- FirebaseException: show "Something went wrong. Please try again."
- Network error: show "No internet connection."
- Form validation: use Form widget with GlobalKey. Call form.validate() before any submit.
- Image upload: if file size > 5MB show "Image too large. Please choose a smaller image."

TASK 5.6: Loading States (apply everywhere)

- Every screen that loads async data: show shimmer placeholders (shimmer package) while loading
- Every button that triggers async action: replace button text with CircularProgressIndicator(strokeWidth:2) while loading. Disable button.
- Pull-to-refresh on feed screen and notifications screen

TASK 5.7: Seed Data for Demo

- Manually add 5 posts to Firestore with realistic Cairo/Egypt locations (use actual lat/lng)
- Add 2 users: one regular, one Super User (karma >= 1000)
- Add 3 comments on one post
- Add 1 DM conversation with a few messages
- This ensures the demo shows a populated, realistic app

TASK 5.8: Screen Recording & Demo Prep

- Install scrcpy for phone mirroring when you have a physical phone: sudo apt install scrcpy
- For emulator: Android Studio built-in screen recorder works
- Demo flow to record: Splash → Login → Feed (scroll) → Post Detail (read comment, edit comment) → Create Post → Map (tap pin) → Chat (send AI message, get response) → Search → Profile (show karma bar) → Settings → Notifications
- Record in landscape or portrait to match your emulator orientation

NOTE: Phase 5 done when: push notifications fire, offline banner shows without network, geofencing triggers local notification, all forms validate, shimmer shows on load. Commit: "feat: phase 5 notifications offline polish"

■ SECTION 4: DAY-BY-DAY TIMELINE

Day	Date	Must Complete By End of Day
1	May 3	Setup done (flutter doctor green, emulator running, Firebase configured, folder structure created)
2	May 4	Phase 1 complete: login, register, Google sign-in, BottomNav, TopBar, Sidebar working
3	May 5	Feed screen shows live Firestore posts. PostCard widget complete. Shimmer loading.
4	May 6	Post Detail complete with real-time comments. Edit/delete comments working.
5	May 7	Create Post working: image upload to Storage, write to Firestore, karma +10.
6	May 8	Map screen with Google Maps, custom markers, filter chips, bottom sheet preview.
7	May 9	Chat list screen and conversation screen working with real-time Firestore messages.
8	May 10	AI Discovery Assistant working: Claude API responding in conversation screen.
9	May 11	Friends sidebar. Chat privacy toggle in settings. Super User karma logic.
10	May 12	Search screen: text search + category filters + trending + recommended for you.
11	May 13	Profile screen complete: tabs, karma progress bar, saved posts with freemium gate.
12	May 14	FCM notifications: request permission, handle foreground + background messages.
13	May 15	Geofencing nearby gem alerts. Offline persistence + connectivity banner.
14	May 16	Error handling on all screens. Loading states. All forms validated. Seed data added.
15	May 17	Full app walkthrough test. Fix any bugs. Record screen recording. Prepare zip.
16	May 18	■ SUBMIT: zip the project folder, attach screen recording, email to professor.

■ SECTION 5: QUICK RULES REMINDER FOR AI AGENT

- ✓ State management = Provider ONLY. No setState for shared data.
- ✓ Every async call = wrapped in try/catch with Snackbar error.
- ✓ Comments/reviews = author can edit and delete. No one else.
- ✓ Super User = karma-based ONLY. Never hardcode isSuperUser=true.
- ✓ Chat privacy = always check chatEnabled before opening DM.
- ✓ AI chatbot = Claude API only. Model = claude-sonnet-4-20250514.
- ✓ Firestore collections = use EXACT names from Section 1. Do not invent new ones.
- ✓ Colors = use constants from app_colors.dart. Do not use raw hex strings elsewhere.
- ✓ Images = always use CachedNetworkImage for network images.
- ✓ Freemium = saved posts capped at 5 for free users. Show upgrade prompt after.
- ✓ Out-of-scope feature 1 = Geofencing (background location alerts).
- ✓ Out-of-scope feature 2 = Offline caching (Firestore persistence + connectivity banner).
- ✓ Do not add Firebase Analytics, AdMob, Stripe or any feature not in this plan.
- ✓ The React/TypeScript Figma Make export is the visual reference. Match its layout in Flutter.